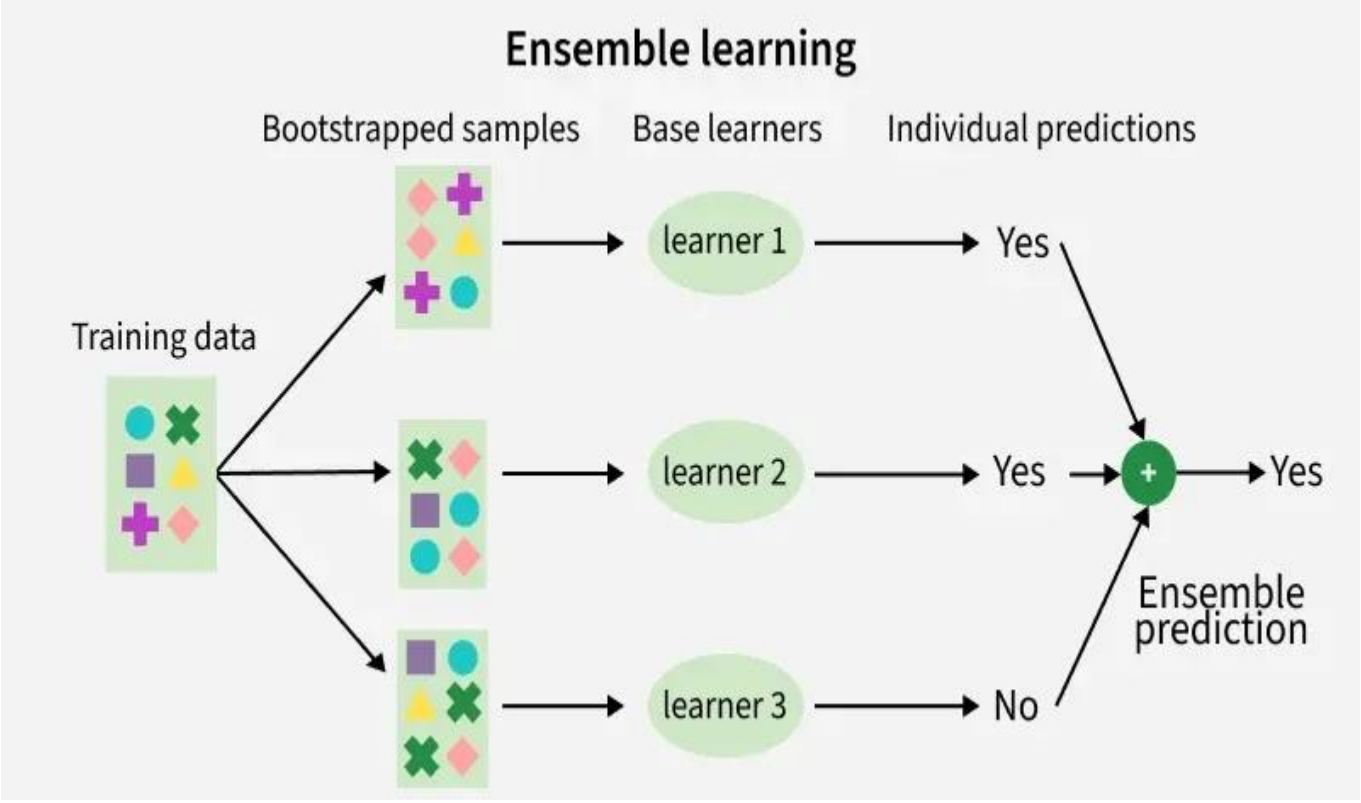
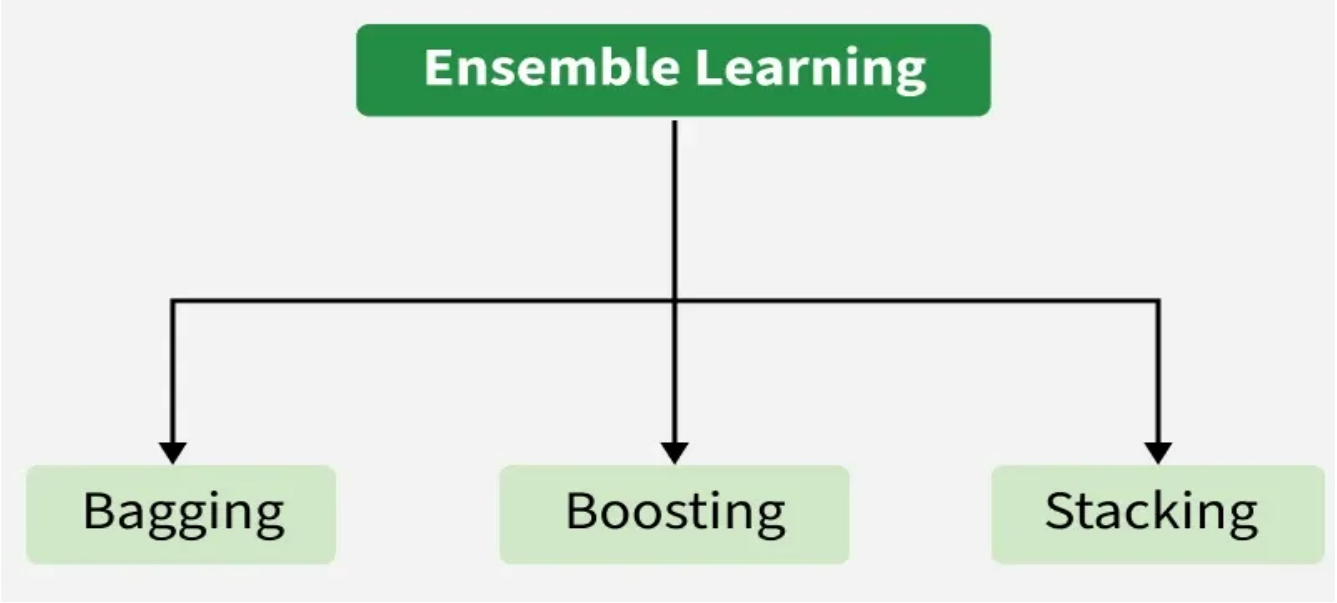
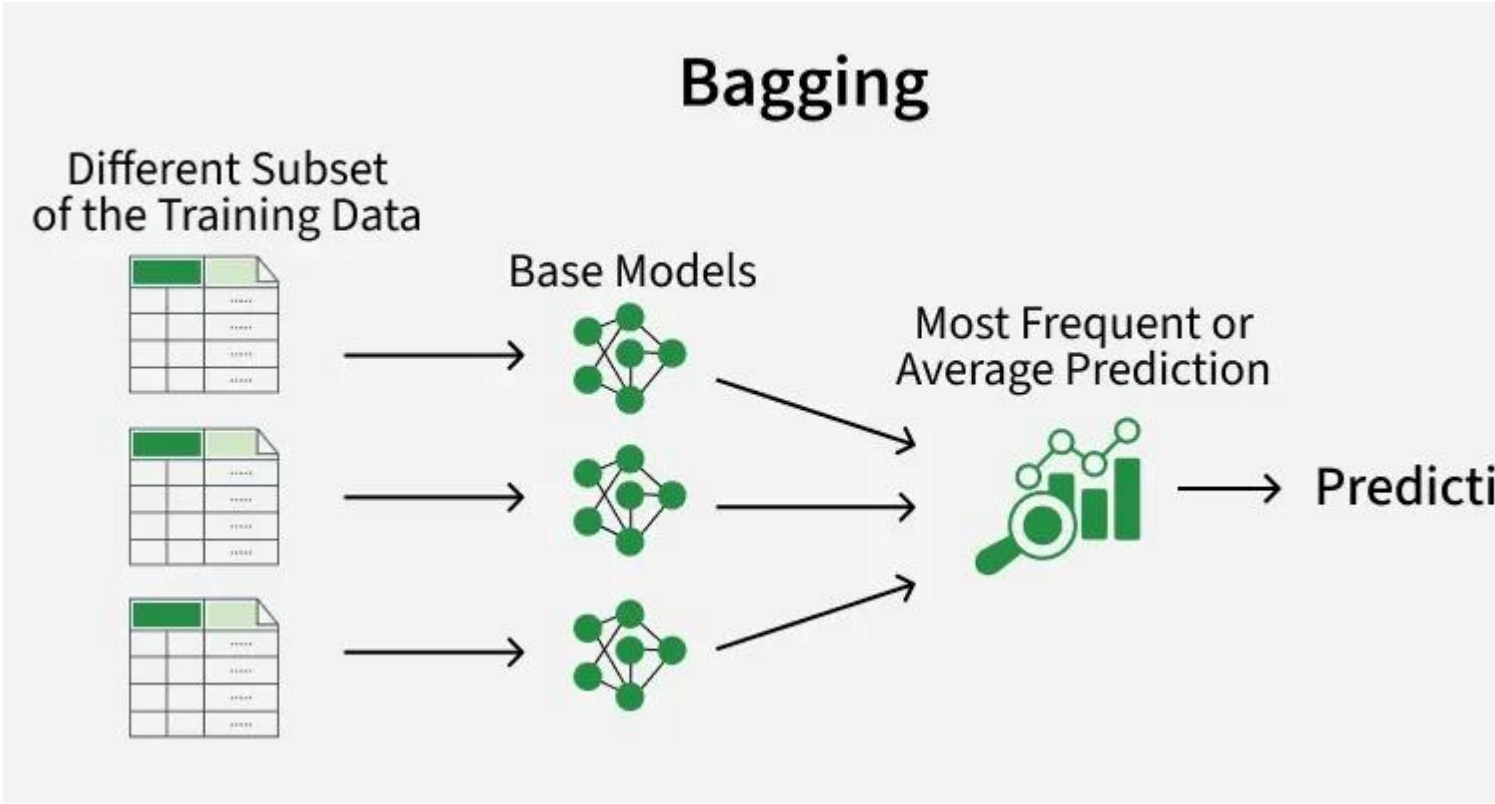
Ensemble Learning

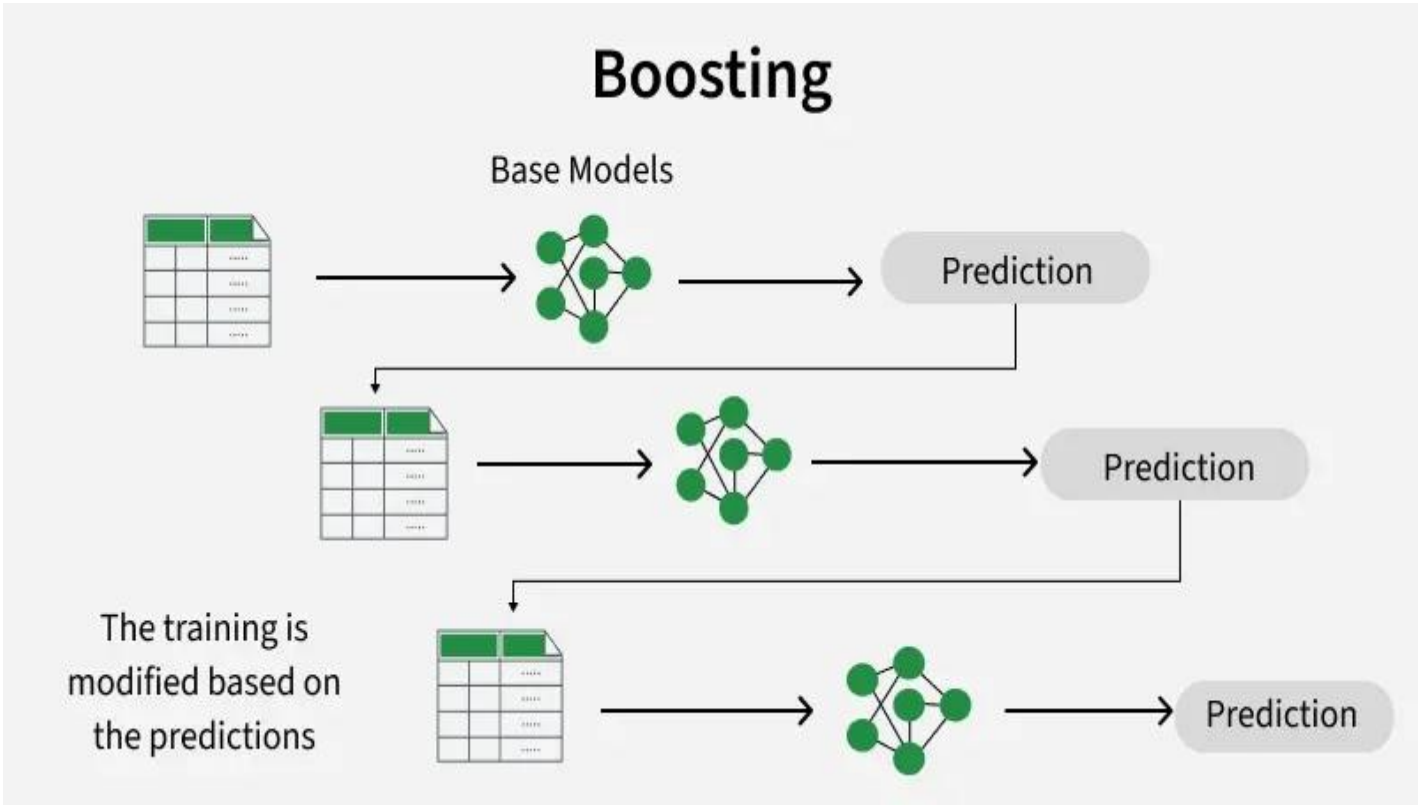
Ensemble learning is a method where multiple models are combined instead of using just one. Even if individual models are weak, combining their results gives more accurate and reliable predictions.

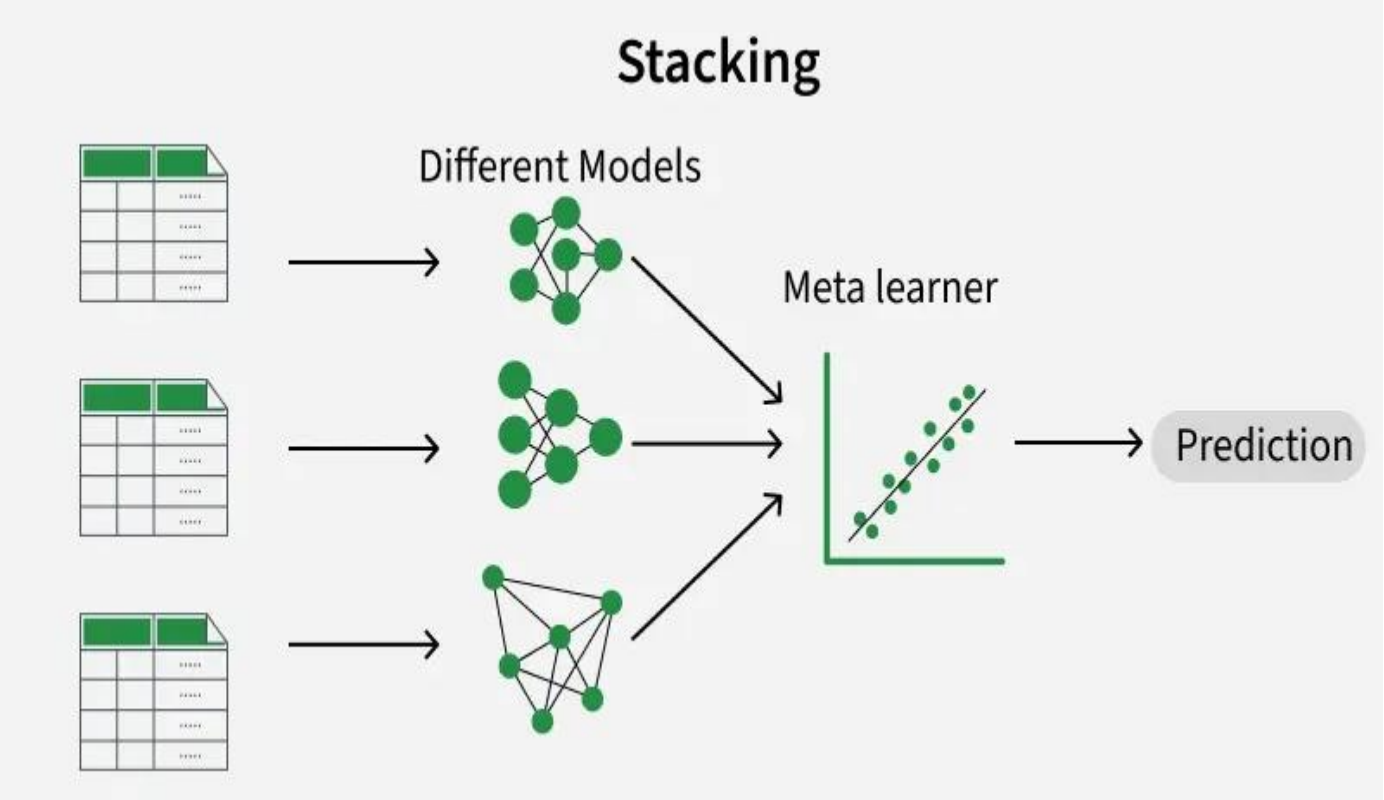
- Uses multiple models together to improve overall accuracy.
- Reduces errors by balancing mistakes across models.
- Works on a simple idea similar to combining opinions from a group.






Bagging
Different Subset of the Training Data
Base Models
Most Frequent or Average Prediction
Predicti


Boosting
Base Models
Prediction
Prediction
The training is modified based on the predictions
Prediction



Types of Ensemble Learning

There are three main types of ensemble methods:

1. **Bagging (Bootstrap Aggregating):** Models are trained independently on different random subsets of the training data. Their results are then combined—usually by averaging (for regression) or voting (for classification). This helps reduce variance and prevents overfitting.
2. **Boosting:** Models are trained one after another. Each new model focuses on fixing the errors made by the previous ones. The final prediction is a weighted combination of all models, which helps reduce bias and improve accuracy.
3. **Stacking (Stacked Generalization):** Multiple different models (often of different types) are trained and their predictions are used as inputs to a final model, called a meta-model. The meta-model learns how to best combine the predictions of the base models, aiming for better performance than any individual model.

- **Bagging Algorithm**

Bagging classifier can be used for both regression and classification tasks. Here is an overview of Bagging classifier algorithm**:**

- **Bootstrap Sampling :** The dataset is divided into multiple subsets by sampling with replacement, creating diverse training data
- **Base Model Training** : A separate model is trained on each subset independently, often in parallel for efficiency

- **Prediction Aggregation** : Predictions from all models are combined using majority voting (classification) or averaging (regression)
- **OOB Evaluation** : Samples not included in a subset are used to evaluate model performance without cross-validation
- **Final Prediction** : The combined output of all models gives a more reliable and accurate result

Implementation

1. Importing Libraries and Loading Data

We will import scikit learn for:

- **BaggingClassifier:** for creating an ensemble of classifiers trained on different subsets of data.
- **DecisionTreeClassifier:** the base classifier used in the bagging ensemble.
- **load_iris:** to load the Iris dataset for classification.
- **train_test_split:** to split the dataset into training and testing subsets.
- **accuracy_score**: to evaluate the model's prediction accuracy.

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

2. Loading and Splitting the Iris Dataset

- **data = load_iris():** loads the Iris dataset, which includes features and target labels.
- **X = data.data:** extracts the feature matrix (input variables).
- **y = data.target:** extracts the target vector (class labels).
- **train_test_split(...):** splits the data into training (80%) and testing (20%) sets, with random_state=42 to ensure reproducibility.

```
data = load_iris()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3. Creating a Base Classifier

Decision tree is chosen as the base model. They are prone to overfitting when trained on small datasets making them good candidates for bagging.

- **base_classifier = DecisionTreeClassifier()**: initializes a Decision Tree classifier, which will serve as the base estimator in the Bagging ensemble.

```
base_classifier = DecisionTreeClassifier()
```

4. Creating and Training the Bagging Classifier

- A **BaggingClassifier** is created using the decision tree as the base classifier.

- **n_estimators = 10** specifies that 10 decision trees will be trained on different bootstrapped subsets of the training data.

```
bagging_classifier = BaggingClassifier(base_classifier, n_estimators=10, random_state=42)
bagging_classifier.fit(X_train, y_train)
```

5. Making Predictions and Evaluating Accuracy

- The trained bagging model predicts labels for test data.
- The accuracy of the predictions is calculated by comparing the predicted labels (y_pred) to the actual labels (y_test).

```
y_pred = bagging_classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Output:

Accuracy: 1.0

2. Boosting Algorithm

Boosting is an ensemble technique where multiple weak models are trained one after another, and each new model focuses on correcting the errors of the previous one to build a strong model. The process works as follows:

- **Initialize Weights** : Start with equal weights for all training data
- **Train Weak Learner** : Train a simple model on the dataset
- **Sequential Learning** : Each new model learns from previous errors
- **Weight Adjustment** : Misclassified samples get higher weights so future models focus more on them

Implementation

1. Importing Libraries and Modules

- **AdaBoostClassifier:** for building the AdaBoost ensemble model.
- **DecisionTreeClassifier:** as the base weak learner for AdaBoost.
- **load_iris:** to load the Iris dataset.
- **train_test_split :** to split the dataset into training and testing sets.
- **accuracy_score:** to evaluate the model's accuracy.

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

2. Loading and Splitting the Dataset

- **data = load_iris():** loads the Iris dataset, which includes features and target labels.

- **X = data.data:** extracts the feature matrix (input variables).
- **y = data.target:** extracts the target vector (class labels).
- **train_test_split(...):** splits the data into training (80%) and testing (20%) sets, with random_state=42 to ensure reproducibility.

```
data = load_iris()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3. Defining the Weak Learner

We are creating the base classifier as a decision tree with maximum depth 1 (a decision stump). This simple tree will act as a weak learner for the AdaBoost algorithm, which iteratively improves by combining many such weak learners.

```
base_classifier = DecisionTreeClassifier(max_depth=1)
```

4. Creating and Training the AdaBoost Classifier

- **base_classifier**: The weak learner used in boosting.
- **n_estimators = 50**: Number of weak learners to train sequentially.
- **learning_rate = 1.0:** Controls the contribution of each weak learner to the final model.
- **random_state = 42:** Ensures reproducibility.

```
adaboost_classifier = AdaBoostClassifier(
    base_classifier, n_estimators=50, learning_rate=1.0, random_state=42
)
adaboost_classifier.fit(X_train, y_train)
```

5. Making Predictions and Calculating Accuracy

We are calculating the accuracy of the model by comparing the true labels y_test with the predicted labels y_pred. The accuracy_score function returns the proportion of correctly predicted samples. Then, we print the accuracy value.

```
y_pred = adaboost_classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Output:

Accuracy: 1.0

Importance

Ensemble learning is a versatile approach that can be applied to machine learning model for:

- **Reduction in Overfitting**: By aggregating predictions of multiple model's ensembles can reduce overfitting that individual complex models might exhibit.

- **Improved Generalization**: It generalizes better to unseen data by minimizing variance and bias.
- **Increased Accuracy**: Combining multiple models gives higher predictive accuracy.
- **Robustness to Noise**: It mitigates the effect of noisy or incorrect data points by averaging out predictions from diverse models.
- **Flexibility**: It can work with diverse models including decision trees, neural networks and support vector machines making them highly adaptable.
- **Bias-Variance Tradeoff**: Techniques like bagging reduce variance, while boosting reduces bias leading to better overall performance.